Symbolic Planning for Automated Manufacturing Using Graphs of Convex Sets and Mixed-Integer Convex Optimization

Louise Hasslöf, StudentID: 314512801

***Abstract*—This project investigates the use of Graphs of Convex Sets (GCS) and Mixed-Integer Convex Programming (MICP) for symbolic task planning in automated manufacturing environments. The original problem is inherently discrete and combinatorial, and aims to find the shortest feasible sequence of operations to reach a goal state. The problem is reformulated by embedding the symbolic state variables into convex sets and expressing operations as convex transition constraints within a GCS framework. To recover the discrete transition sequences, the GCS formulation is reformulated as an MICP, by introducing binary edge activation variables and indicator constraints. The implementation was done in MATLAB using the MOSEK solver and was evaluated on two simple examples and compared to Dijkstra's algorithm as a benchmark. The obtained solutions from the GCS/MICP formulation for the test cases were all consistent with the results from Dijkstra. The examples used were small in scale, but the result demonstrate the feasibility of the proposed formulation for symbolic planning, and provides a foundation for further investigation of larger manufacturing systems.**

I. Introduction

Symbolic task planning for automated manufacturing environments requires determining feasible sequences of operations that transform an initial system state into a desired goal state. This is an inherently discrete, combinatorial, and non-convex problem. The number of possible operations sequences grows combinatorially as the number of resources, operations, and state variables increases, which motivates investigation of approaches that can exploit the problem structure to compute feasible plans, without needing explicit enumeration of all possible states.

In this work, operations are treated as black boxes, providing no explicit information about time, energy, or internal cost functions, making our objective to minimize the length of the operation sequence from an initial state to a goal state. The objective can thus be formulated as:

$$\min_{x \in P(s_0, s_g)} |x| \tag{1}$$

where *x* is the operation sequence, $P(s_0, s_g)$ is the set of all feasible sequences from an initial state s_0 to a goal state s_g, and $|x|$ is the length of the sequence. The planning problem can be modelled as a directed graph, where states correspond to nodes and operations correspond to feasible transitions. After the problem has been solved using the proposed method, an implementation of Dijkstra's algorithm is used as a lower-bound baseline for comparison with the proposed formulation.

II. Background

A. States and Operations

The model for the control system used for this project is based on [1] and [2], and a compressed summary is presented here: **State** $\mathcal{S} = \{\langle v_i, x_i \rangle\}$ contains all necessary information about the system. **Guard** is a Boolean function over a state, $g : \mathcal{S} \to \{True, False\}$. **Action** initiates state changes and initializes/resets resources an operation requires/required to execute. $a : \mathcal{S} \to \mathcal{S}$. It invokes the mechanism responsible for executing the operation. A **Transition** is a guard-action pair, that is enabled only when their guard evaluates as true, allowing the action to be triggered. Transitions are used to update states. **Preconditions** and **postconditions** are both transitions with distinct purposes; a precondition initialises an operation and a postcondition concludes it. An **operation** represents a time consuming task, and is defined by a precondition and a postcondition. When executed, it performs a low-level goal. The guard of the precondition determines if the operation can start, while its action updates the state to ensure safe operation. The postcondition's guard signals completion .

In general, an operation may have multiple postconditions representing different possible outcomes, such as success or failure, yielding a non-deterministic transition model. Due to scope restrictions, this project is restricted to a deterministic transition model where each operation only has a single postcondition.

Figure 1, illustrates a very simple system controlling a door. It contains four states, but only three are meaningful, and it contains four operations that change the state.

Important assumptions to note regarding this project: (i) Operations are deterministic, (ii) The transition model is captured by a directed graph with a finite number of states. (iii) The system is fully observable. (iv) All state variables can be embedded as convex sets.

B. Graphs of Convex Sets (GCS)

A Graphs of Convex Sets (GCS) is a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where each vertex $v \in \mathcal{V}$ is associated with a convex constraint set $\chi_v \subset \mathbb{R}^{n_v}$, local optimization variables $x_v \in \mathbb{R}^{n_v}$, and a cost function $f : \mathbb{R}^{n_v} \to \mathbb{R}$. Each edge $e = (v, w) \in \mathcal{E}$ introduce additional convex costs and constraints linking adjacent vertices, defined over $\chi_e \subseteq \mathbb{R}^{n_v + n_w}$ with cost $f : \mathbb{R}^{n_v + n_w} \to \mathbb{R}$[3] [4].

Since GCS formulation contains both continuous and discrete components, the resulting optimization problem is mixed

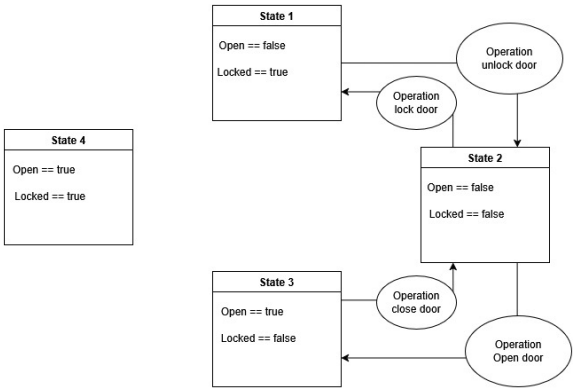

Fig. 1: A simple state diagram for opening/closing and locking/unlocking a door.

[3]. A GCS optimization problem can be formulated as follows:

$$\text{minimize} \sum_{v \in \mathcal{W}} f_v(x_v) + \sum_{e=(v,w) \in \mathcal{F}} f_e(x_v, x_w) \tag{2a}$$

$$\text{subject to } H = (\mathcal{W}, \mathcal{F}) \in \mathcal{H} \tag{2b}$$

$$x_v \in \chi_v \qquad \forall v \in \mathcal{W} \tag{2c}$$

$$(x_v, x_w) \in \chi_e \qquad \forall e = (v, w) \in \mathcal{F} \tag{2d}$$

where H is a discrete subgraph of the original graph. Each vertex v is associated with a convex set $\chi_v \subseteq \mathbb{R}^{n_v}$ and continuous variable $x_v \in \chi_v$, while each edge $e = (v, w)$ is associated with a convex set χ_e that constrains the pair of variables (x_v, x_w). The functions $f_v : \chi_v \to \mathbb{R}$ and $f_e : \chi_e \to \mathbb{R}$ are convex vertex and edge cost functions. The optimization problem thus determines the feasible subgraph and continuous variables that minimize the total accumulated cost[3].

C. Mixed-Integer Convex Optimization (MICP)

A mixed-integer convex program (MICP) is an optimization problem that involves both discrete and continuous decision variables, where the objective function and constraints are convex for the continuous variables [5]. It can be viewed as a subclass of mixed-integer programs (MIPs) that incorporates convex structure into the continuous part of the model [3]. An MICP can be formulated as follows:

$$\text{minimize } f(x, y) \tag{3a}$$

$$\text{s.t. } g_i(x, y) \leq 0, i = 1, \ldots, m \tag{3b}$$

$$x \in \mathbb{R}^n, y \in \mathbb{Z}^p \tag{3c}$$

where x are continuous variables and y are discrete/integer variables. Functions $f : \mathbb{R}^n \times \mathbb{Z}^p \to \mathbb{R}$ and $g_i : \mathbb{R}^n \times \mathbb{Z}^p \to \mathbb{R}$ are convex in x for every fixed y, meaning the resulting continuous subproblems are convex[5], [6]. In general, MICP problems are NP-hard due to the discrete variables [3].

TABLE I: The state variables used for the simplified simple manufacturing environment, which only have 36 theoretical states.

AGV1_pos $\in \{0, 1, 2, 3\}$
R1_pos $\in$ {"ConveyorBelt", "buffer", "0"}
item_pos $\in$ {"ConveyorBelt", "R1", "AGV1"}

III. Problem description and Modelling

The original system is modelled as a set of states with operations defining the transitions between them. The proposed formulation should be viewed as a convex relaxation of the original problem. By embedding the discrete symbolic states into convex representations, the resulting formulation enables the use of convex optimization techniques while approximating the underlying discrete transition structure.

To motivate the formulation, consider a simple manufacturing environment consisting of two Automated Guided Vehicles (AGVs) and one stationary robotic arm. These three robots need to collaborate to transport an item from a conveyor belt to different workstations. For systems like this, the objective is to minimize the length of the operation sequence defined in (1). The state variables used in this example are listed in Appendix III. This can be modelled as a directed graph, similar to the example in Figure 1. The model has an upper bound of $|\mathcal{S}| = 20480$ unique theoretical states, but a lot of them are unreachable, undesirable, or have no physical meaning, like state 4 in Figure 1. The available operations include moving each robot to each of their possible positions and actuating the robotic gripper.

Explicit enumeration scales combinatorially with the number of resources and state variables, as noted even the very simplified manufacturing example has more than 20,000 theoretical states. This motivates the use of convex relaxation and convex optimization approach, since it avoids explicit enumeration of all feasible operation sequences and instead exploits a structured optimization framework to handle the combinatorial complexity by using modern convex optimization techniques.

Implementing this complete, but simple, manufacturing environment is beyond the scope of this project, and instead a reduced benchmark that preserves the essential characteristics of the problem will be solved instead. The state variables of the reduced version can be seen in Table I and further derivations and details are provided in Appendix C.

A. States and Operations to GCS

The first step in addressing the optimization problem in Equation (1) is to rewrite the symbolic planning problem as a GCS formulation. States are represented as convex vertex sets, operations as directed edges, guards as feasibility constraints, actions as state transition constraints. GCS formulations operate over convex sets, meaning the discrete variables need to be embedded into a continuous space. To illustrate the process, I first demonstrate how Boolean state variables can be handled using the example in Figure 1, before extending the approach to other data types.

1) Handling Boolean State Variables: **Embed the Boolean states into** $\mathbb{R}^n$, or for the door example, $\mathbb{R}^2$. Define a continuous vector: $x_i = (o, l) \in \mathbb{R}^2$, where $o \approx$ Open and $l \approx$ Locked. Open = False corresponds to $o = 0$ etc.

Define convex sets for the symbolic states, let:
State 1 $\rightarrow X_1 = \{x | o = 0, l = 1\}$, State 2 $\rightarrow X_2 = \{x | o = 0, l = 0\}$, State 3 $\rightarrow X_3 = \{x | o = 1, l = 0\}$

These sets are degenerate convex sets (singletons), meaning they do not provide meaningful relaxation in this case, since they force the variables to essentially take the same values as the original symbolic representation. Instead, they are further relaxed into small bounded convex neighbourhoods, where the variables can vary continuously within a small convex interval around the symbolic states. This yields non-degenerate convex sets that are more suitable for optimization and provides explicit continuous relaxation of the state space. Therefore, we rewrite States 1 and 2 as: State 1: $X_1 = \{x | o \in [0, \epsilon], l \in [1 - \epsilon, 1]\}$, State 2: $X_2 = \{x | o \in [0, \epsilon], l \in [0, \epsilon]\}$, for some fixed parameter $\epsilon \in (0, 1)$. More generally, each relaxed state can be written in polyhedral form: $X_i = \{x_i | A_i x_i \leq b_i\}$ where each row of $A_i x_i \leq b_i$ defines a halfspace . This forms a polyhedron, which is a convex set, therefore X_i is convex [6]. For example, X_1 the matrix version can be written as $A_1 x_1 \leq b_1 \Leftrightarrow$

$$\begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} o_1 \\ l_1 \end{bmatrix} \leq \begin{bmatrix} \epsilon \\ 0 \\ 1 \\ \epsilon - 1 \end{bmatrix} \tag{4}$$

The same matrix A can be used for all states:

$$X_i = \{x_i | A x_i \leq b_i\} \tag{5}$$

It is important to note that these convex relaxations are only approximations, since they may admit intermediate continuous values that do not correspond to realizable symbolic states, which may introduce a relaxation gap.

Moreover, unlike this simple door example, where each state is associated with an individual convex set X_i, applying the same construction for the manufacturing environment would require explicit enumeration of states, which is not desirable. For the reduced manufacturing example it would correspond to 36 convex sets, while for the complete model in IV would require 20,480 convex sets. Instead a shared convex embedding of the state variables is used, where each state variable is first embedded separately, using the convex hull described in section III-A2. Afterwards it is stacked into a single convex state representation.

The extreme points of each embedding correspond to exactly one of the original one-hot vectors, described in section III-A2 meaning the extreme points correspond to combinations of the symbolic assignments. Consequently, the reduced manufacturing example has $4 \times 3 \times 3 = 36$ extreme points, matching the 36 theoretical symbolic states. The operation constraints and flow conservation conditions determine which states are feasible, eliminating the need for a separate reachability analysis. An example for the reduced manufacturing example can be seen in appendix C1.

Define edges as convex constraints: Each operation defines a feasible transition relation between states, meaning the operations need to be redefined as convex constraints. An edge, for this example, (x_i, x_j) belongs to $\mathbb{R}^2 \times \mathbb{R}^2$. For example, defining the edge between X_1 and X_2, can be done as follows: $\chi_{12} = \{(x_i, x_j) | x_i \in X_1, x_j \in X_2, o_i = o_j\}$ or from X_2 to X_3: $\chi_{23} = \{(x_i, x_j) | x_i \in X_2, x_j \in X_3, l_i = l_j\}$. We can also generally write this in matrix form as:

$$\chi_{ij} = \{(x_i, x_j) | A x_i \leq b_i, A x_j \leq b_j, B_{ij} x_i + C_{ij} x_j = d_{ij}\} \tag{6}$$

where B_{ij}, C_{ij}, and d_{ij} encode the specific transition logic between the two states, while the inequalities ensure start and end states are valid (node constraints). Important to note that some state variables are invariant across an edge while others intentionally are modified. For example, the edge between X_1 and X_2 is illustrated in Appendix B1. In this case, openness stays the same, while locked changes.

For each directed edge,$(x_i, x_j) \in \mathcal{E}$ a **flow variable is added**: $\phi_{ij} \in [0, 1]$, where $\phi_{ij} = 1$ indicates that the transition is active (flow from node i to j is active), while $\phi_{ij} = 0$ disables the edge. It represents flow along edges, but one key aspect of the GCS is that the edge constraint should only matter if an edge is active [3]. Let the edge set be defined as:

$$\chi_{ij} = \left\{ (x_i, x_j) | F_{ij} \begin{bmatrix} x_i \\ x_j \end{bmatrix} \leq g_{ij} \right\} \tag{7}$$

where F_{ij} is a matrix of constraint coefficients and g_{ij} is the corresponding right-hand-side vector. The rewritten constraint of equation (16) can be seen in Appendix B1. The edge activation condition can be written as $\phi_{ij} > 0 \Rightarrow (x_i, x_j) \in \chi_{ij}$. The convex relaxation can then be written as:

$$F_{ij} \begin{bmatrix} x_i \\ x_j \end{bmatrix} \leq \phi_{ij} g_{ij} \tag{8}$$

but it is important to note that this may admit fractional edge activations. However later when reformulating the problem as MICP, binary edge activation variables will be reintroduced to recover the discrete transition selection. From here we can also write the standard flow conservation constraint:

$$\sum_j \phi_{ij} - \sum_k \phi_{ki} = \begin{cases} 1, i = s_0 \\ -1, i = s_g \\ 0, \text{ otherwise} \end{cases} \tag{9}$$

where the first term represent total outgoing flow from node i and the second term represents total incoming flow. The constraint enforces flow conservation throughout the graph, since every intermediate node must pass on exactly the amount of flow it receives, while the source node s_0 generates one unit of flow and the goal node s_g absorbs one unit. Consequently, any feasible solution corresponds to a continuous path from the source to the goal. This allows us to write our new objective as minimizing the total flow through the graph:

$$\min \sum_{(i,j) \in \mathcal{E}} \phi_{ij} \tag{10}$$

GCS formulation: To summarize, we have optimization variables, $x_i \in X_i, \forall i \in \mathcal{V}$, representing the continuous

embedding of node states, and $\phi_{ij} \in [0,1], \forall (i,j) \in \mathcal{E}$ representing flow on the directed edges. Using the other parts introduced above, the boolean door example can be expressed as the following GCS optimization problem:

$$\min_{x,\phi} \sum_{(i,j)\in\mathcal{E}} \phi_{ij} \tag{11a}$$

$$\text{s.t.} \sum_{j\in\mathcal{I}_i^{out}} \phi_{ij} - \sum_{k\in\mathcal{I}_i^{in}} \phi_{ki} = \begin{cases} 1, i = s_0 \\ -1, i = s_g \\ 0, \text{ otherwise} \end{cases} \quad \forall i \in \mathcal{V} \tag{11b}$$

$$A_i x_i \leq b_i, \quad \forall i \in \mathcal{V} \tag{11c}$$

$$F_{ij} \begin{bmatrix} x_i \\ x_j \end{bmatrix} \leq \phi_{ij}\, g_{ij}, \quad \forall (i,j) \in \mathcal{E} \tag{11d}$$

$$0 \leq \phi_{ij} \leq 1, \quad \forall (i,j) \in \mathcal{E} \tag{11e}$$

where $\mathcal{I}_v^{out} = \{(v,w) \in \mathcal{E}\}$ and $\mathcal{I}_v^{in} = \{(w,v) \in \mathcal{E}\}$ are outgoing and ingoing edges for a vertex $v \in \mathcal{V}$. Since the set of vertices are convex polyhedra, the edge constraints are affine, and the flow conservation constraints are linear, the relaxed optimization problem is a convex optimization problem. The reason to why we will not directly solve equation (11) is because it is a *relaxation* and will not yield a discrete valid sequence of operations, but instead a convex combination of paths. The problem lies in (11e) (the flow variables $\phi_{ij} \in [0,1]$), since fractional values are possible which allows the flow to split. A flow like $\phi_{12} = 0.5$ and $\phi_{13} = 0.5$ is possible with this GCS formulation. Reformulating the GCS as an MICP formulation will explicitly enforce the discrete edge selection, and thus a discrete path through the relaxed state-space can be found.

2) Handling Other Data-Types as State Variables: Example 1 focuses exclusively on boolean state variables, but practical applications, and the model used for this project also uses state variables of other types, like integers or strings. The following outlines how to incorporate such data types within the same framework.

Strings/symbolic state variables: By using one-hot encoding, categorical state variables can be rewritten in canonical basis vector form. Let symbolic variable *S* take on *n* discrete unique values. Each value can then be represented by canonical basis vectors in $\mathbb{R}^n$:

$$\mathcal{Z} = \left\{ z \in \{0,1\}^n \middle| \sum_{i=1}^{n} z_i = 1 \right\} \tag{12}$$

The requirement that exactly one component satisfies ($z_i = 1$) ensures that each symbolic value is represented uniquely. This is preferable to multi-hot encodings, which may introduce artificial additive relationships between categories. For example, consider the symbolic variable: R1_pos$\in$ {*aboveConveyorBelt, aboveAGV1, aboveAGV2*}. If the values are encoded as: $aboveConveyorBelt = [1,0], aboveAGV1 = [0,1],\ aboveAGV2 = [1,1]$, then $[1,0]+[0,1] = [1,1]$, which implies that the state aboveAGV2 can be interpreted as the combination of the other two states. This is generally false for the underlying system, as the symbolic variables are generally mutually exclusive. A convex relaxation of the one-hot discrete set of state variables is given by the convex hull of the set:

$$\text{conv}(\mathcal{Z}) = \left\{ z \in [0,1]^n \middle| \sum_{i=1}^{n} z_i = 1 \right\} \tag{13}$$

which forms a probability simplex of the one-hot vectors [6]. There are other alternative encodings one can do to get a more compact representation, like for example multi-hot encoding or binary encoding, but geometrically the simplex provides a clean and tight convex relaxation that is easy to interpret and constrain.

Introducing additional state variable types does not fundamentally alter the GCS formulation (equation (11)) - only the convex embedding of the variables changes. Once rewritten in convex representation, the same method for node, edge, and flow formulations can be applied.

Integers can generally be handled in two ways depending on their interpretation. If the integers represent categories, like for example 0 = idle, 1 = moving, 2 = busy, they can be treated like categorical variables using one-hot encoding. If the integers instead represent physical numeric quantities, such as battery level or object count, they can be relaxed directly: $n \in \{0,1,...,m\} \rightarrow n \in [0,m]$. This preserves convexity, but may admit to non-physical intermediate solutions.

B. GCS to MICP

There are multiple ways of obtaining an MICP formulation. Marcucci [3] derives a strong MICP through an intermediate MINCP formulation and perspective reformulations, but to keep the formulation and implementation of this project within the scope, a simpler approach which may admit to weaker relaxation is used instead.

As discussed in Section III-A1 the GCS formulation permits fractional flow values. To recover discrete paths, the flow variables are restricted to $\phi_{ij} \in \{0,1\}$. This transforms the relaxed GCS formulation into a mixed-integer optimization problem. The remaining challenge is ensuring that the edge constraints are only activated when the corresponding edge is selected. This project tackles that problem by using indicator constraints for each edge (i,j) :

$$\phi_{ij} = 1 \Rightarrow F_{ij} \begin{bmatrix} x_i \\ x_j \end{bmatrix} \leq g_{ij} \tag{14}$$

which is enforced directly by the MICP solver. Compared to the stronger perspective reformulations proposed by Marcucci [3], this approach yields a weaker relaxation but substantially reduces formulation complexity, which is appropriate given the scope limitations. The resulting MICP formulation is therefore identical to the GCS formulation, with the exception that Equation (11e) is replaced by:

$$\phi_{ij} \in \{0,1\} \quad \forall (i,j) \in \mathcal{E} \tag{15}$$

and equation (11d) is replaced with the indicator constraint in equation (14).

1) Optimality of the MICP: The final MICP formulation is equivalent to a unit-cost shortest path problem, where the binary edge activation variables together with the flow conservation constraints ensure that every feasible solution corresponds to a discrete path between source and goal nodes. Since the objective counts the number of selected operations, minimizing the objective gives a shortest operation sequence.

Example	MICP	Dijkstra	Dijkstra solve time	Match
Simple Door $1 \rightarrow 3$	2	2	0.0009 s	Yes
Simple Door $3 \rightarrow 2$	1	1	0.0002 s	Yes
Reduced Manufacturing: $[1,2,1] \rightarrow [3,2,3]$	8	8	0.0122 s	Yes
Reduced Manufacturing: $[3,2,1] \rightarrow [1,1,3]$	10	10	0.0024 s	Yes

TABLE III: Comparison of optimal objective values obtained by the proposed MICP formulation and the Dijkstra baseline.

Furthermore, any optimal solution is guaranteed to be cycle-free, since all edge costs are positive. Therefore, the final MICP formulation recovers a shortest feasible symbolic plan. In contrast, the relaxed GCS formulation permits fractional edge activations and therefore generally recovers convex combinations of paths rather than feasible symbolic plans.

IV. Results

The model was implemented in MATLAB using the MOSEK solver. The complete code is provided on GitHub[1].

Table II displays results for the proposed MICP solution, while Table III shows a comparison between the proposed MICP formulation and the baseline Dijkstra implementation .The proposed MICP formulation successfully solved all test cases and returned the same feasible symbolic paths as the Dijkstra baseline in every case.

Example	Path	Solvetime
Simple Door $1 \rightarrow 3$	$1 \rightarrow 2 \rightarrow 3$	0.2979 s
Simple Door $3 \rightarrow 2$	$3 \rightarrow 2$	0.2378 s
Reduced Manufacturing: Move the item from conveyor belt to position 3 $[0,2,1] \rightarrow [3,2,3]$	$4 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow 24 \rightarrow 33$	0.5389 s
Reduced Manufacturing move the item from conveyor belt to position 1 $[3,2,1] \rightarrow [1,1,3]$	$31 \rightarrow 28 \rightarrow 19 \rightarrow 20 \rightarrow 23 \rightarrow 26 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow 15 \rightarrow 12$	0.6874 s

TABLE II: Four test cases - Shortest paths and solver runtimes obtained using the proposed MICP formulation.

V. Discussion

The Dijkstra baseline recovered the same optimal operation sequence and objective values as the proposed MICP formulation. However, the benchmark problems considered were small, making explicit graph construction feasible and allowing Dijkstra's algorithm to efficiently compute shortest paths. For these examples, the MICP formulation offers no clear computational advantage over Dijkstra. The motivation to use the proposed formulation is not improved performance on smaller graphs, but rather the ability to represent the planning problem through constraints without requiring explicit enumeration of the complete graph.

As the number of resources, symbolic states, and operations increases, the state space grows combinatorially, making explicit graph construction impractical. The combinatorial growth in problem size is a limitation of explicit graph-search algorithms which motivates alternative approaches. The proposed formulation provides a structured optimization framework that could, in principle, exploit convexity and modern solvers to solve the problem without explicit enumeration. Explicit graph enumeration was used in the implementation to enable easy comparison with Dijkstra, but the underlying optimization framework of the MICP is not inherently restricted to it.

The agreement between Dijkstra and the MICP serves as validation of the proposed method, while further investigation of larger-scale problems is required to assess its scalability.

A. Implementation details

During the implementation phase, alternative frameworks like YALMIP [7] were briefly considered. While being similar to CVX, YALMIP has the benefit of allowing high-level abstractions for logical implications, meaning Equation (14) can be expressed as **implies(phi(e),F*z <= g)**[8]. However, due to project constraints, the implementation was carried out in CVX. Since CVX does not support indicator constraints, the corresponding constraints were implemented using an equivalent Big-M reformulation.

No issues attributable to the Big-M reformulation were observed in the tested cases, but further work could investigate direct implementations of indicator constraints or stronger formulations, such as the perspective reformulations proposed by Marcucci [3], which may yield tighter relaxations and improved numerical robustness.

VI. Conclusion

This project investigated the use of Graphs of Convex Sets (GCS) and a Mixed-Integer Convex programming (MICP) formulation for symbolic task planning for manufacturing environments. This work demonstrates that symbolic planning problems in manufacturing environments can be reformulated within the GCS framework and solved through MICP while preserving optimal symbolic plans on benchmark examples. The proposed formulation was implemented using MATLAB and MOSEK and evaluated on only two simple planning examples, but the formulation provides a foundation for further investigation of optimization-based planning approaches on larger-scale systems.

Results show that the proposed formulation is able to obtain the same optimal operation paths as Dijkstra's Algorithm. Due to the limited scope of this project, further work is needed to test the scalability and implementation on larger scale scenarios.

[1]https://github.com/potaaaat/SymbolicPlanner_using_GCS_and_MICP

SOURCES

[1] M. Dahl, E. Erős, K. Bengtsson, M. Fabian, and P. Falkman, "Sequence planner: A framework for control of intelligent automation systems," *Applied Sciences*, vol. 12, no. 11, 2022, ISSN: 2076-3417. DOI: 10.3390/app12115433. [Online]. Available: https://www.mdpi.com/2076-3417/12/11/5433.

[2] L. Hasslöf, F. Högström, A. Leion, and T. Taraev, "Developing a ROS2 infrastructure and control system," Department of Electrical Engineering, Bachelor Thesis, Chalmers University of Technology, Gothenburg, Sweden, 2025. [Online]. Available: https://hdl.handle.net/20.500.12380/309571.

[3] T. Marcucci, "Graphs of convex sets with applications to optimal control and motion planning," Ph.D. dissertation, Massachusetts Institute of Technology, Cambridge, MA, USA, May 2024. [Online]. Available: https://dspace.mit.edu/handle/1721.1/156598.

[4] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM Journal on Optimization*, vol. 34, no. 1, pp. 507–532, 2024. DOI: 10.1137/22M1523790. eprint: https://doi.org/10.1137/22M1523790. [Online]. Available: https://doi.org/10.1137/22M1523790.

[5] D. Axehill, L. Vandenberghe, and A. Hansson, "Convex relaxations for mixed integer predictive control," *Automatica*, vol. 46, no. 9, pp. 1540–1545, 2010, ISSN: 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2010.06.015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0005109810002657.

[6] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.

[7] J. Lofberg, "Yalmip : A toolbox for modeling and optimization in matlab," in *2004 IEEE International Conference on Robotics and Automation (IEEE Cat. No.04CH37508)*, 2004, pp. 284–289. DOI: 10.1109/CACSD.2004.1393890.

[8] J. Löfberg, *Command reference: Implies*, https://yalmip.github.io/command/implies/, Accessed: June 12, 2026, 2016.

APPENDIX

A. The state variable for the simple manufacturing example in section III:

TABLE IV: The state variables used for the simple manufacturing environment.

AGV1_pos $\in \{0, 1, 2, 3\}$
AGV2_pos $\in \{0, 1, 4, 5\}$
AGV1_status $\in$ {"idle", "busy"}
AGV2_status $\in$ {"idle", "busy"}
R1_pos $\in$ {"aboveConveyorBelt", "aboveAGV1", "aboveAGV2", "buffer"}
R1_gripper_closed $\in \{$true, false$\}$
AGV1_tray_empty $\in \{$true, false$\}$
AGV2_tray_empty $\in \{$true, false$\}$
item_pos $\in$ {"ConveyorBelt", "R1", "AGV1", "AGV2", $0, 1, 2, 3, 4, 5\}$

B. Derivations

1) Deriving χ_{12} with flow variables for the simple door example: The goal is combine the inequalities and equalities between edges. For example, the edge between X_1 and X_2 can be written in matrix form as:

$$\chi_{12} = \Big\{(x_1, x_2) \;\Big|\; Ax_1 \leq b_1,\ Ax_2 \leq b_2, \quad \begin{bmatrix}1 & 0\end{bmatrix}\begin{bmatrix}o_1 \\ l_1\end{bmatrix} + \begin{bmatrix}-1 & 0\end{bmatrix}\begin{bmatrix}o_2 \\ l_2\end{bmatrix} = 0\Big\} \tag{16}$$

We want to combine to one inequality. The equality constraint $\begin{bmatrix}1, 0\end{bmatrix}\begin{bmatrix}o_1 \\ l_1\end{bmatrix} + \begin{bmatrix}-1, 0\end{bmatrix}\begin{bmatrix}o_2 \\ l_2\end{bmatrix} = 0$ preserves the relation $o_1 = o_2$, which we can rewrite as $o_1 - o_2 = 0$. This can be written as two inequalities that together imply equality: $o_1 - o_2 \leq 0$ and $-(o_1 - o_2) = -o_1 + o_2 \leq 0$. We have vector $z = \begin{bmatrix}o_1 \\ l_1 \\ o_2 \\ l_2\end{bmatrix}$ so multiplying it with $\begin{bmatrix}1 & 0 & -1 & 0 \\ -1 & 0 & 1 & 0\end{bmatrix}$ preserves the implication of $o_1 = o_2$. This mean we now only has inequalities for all constraints and they all can be combined into one big constraint matrix:

$$\chi_{12} = \left\{ z \;\middle|\; \begin{bmatrix}A & 0 & & \\ 0 & A & & \\ 1 & 0 & -1 & 0 \\ -1 & 0 & 1 & 0\end{bmatrix} z \leq \begin{bmatrix}b_1 \\ b_2 \\ 0 \\ 0\end{bmatrix} \right\} = \left\{ z \;\middle|\; \begin{bmatrix}1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & -1 & 0 \\ -1 & 0 & 1 & 0\end{bmatrix} \begin{bmatrix}o_1 \\ l_1 \\ o_2 \\ l_2\end{bmatrix} \leq \begin{bmatrix}\epsilon \\ 0 \\ 1 \\ \epsilon - 1 \\ \epsilon \\ 0 \\ \epsilon \\ 0 \\ 0 \\ 0\end{bmatrix} \right\} \tag{17}$$

C. Operations for the simplified simple manufacturing example

The example in I has 36 theoretical possible states and 12 possible operations described in Table V. The states and operations describe an environment where a robot arm can pick

up an item from a conveyor belt and put it down on an AGV that then can transport the item to a different workstation.

Operation	Variations	Precondition	Postcondition
Move_AGV1	$0 \to 1$, $1 \to 0$ $0 \to 2$, $2 \to 0$ $2 \to 3$, $3 \to 2$	AGV1_pos $\neq$ goal_position	AGV1_pos = goal_position
Move_R1	cb $\to$ buffer, buffer $\to$ 0, 0 $\to$ buffer	R1_pos $\neq$ goal_position	R1_pos = goal_position
Pick_up_item	cb $\to$ R1	item_pos = cb AND R1_pos = cb	item_pos = R1
Drop_item	R1 $\to$ AGV1	item_pos = R1 AND R1_pos = 0 AND AGV1_pos = 0	item_pos = AGV1

TABLE V: The 12 operations for the simplified simple manufacturing example. As can bee seen the item can only be picked up from the conveyor belt (cb) and is transfered to R1, and the item can only be dropped at position 0 if it currently is in grabed by the robot.

1) Example of discrete the symbolic variables for the reduced manufacturing example: Like described in Section III-A2,the discrete symbolic variable for, for example R1_pos, can be described as:

$$Z_{R1} = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

Its convex relaxation is the convex hull:

$$\text{conv}(Z_{R1}) = \left\{ z^{R1} \in [0,1]^3 \;\middle|\; \mathbf{1}^T z^{R1} = 1 \right\}.$$

The same was done for Item_pos and AGV1_pos.

To describe the state of the system. like explained in section III-A1, the embedded variables are first stacked: $x = \begin{bmatrix} z^{AGV1} \\ z^{R1} \\ z^{Item} \end{bmatrix}$ and then combined into a convex polytope:

$$x = \begin{bmatrix} z^{AGV1} \\ z^{R1} \\ z^{item} \end{bmatrix}.$$

where:

$$\begin{aligned} z^{AGV1} &\in [0,1]^4, \quad \sum_{i=1}^{4} z_i^{AGV1} = 1, \\ z^{R1} &\in [0,1]^3, \quad \sum_{i=1}^{3} z_i^{R1} = 1, \\ z^{item} &\in [0,1]^3, \quad \sum_{i=1}^{3} z_i^{item} = 1. \end{aligned} \tag{18}$$